Information Web Retrieval and Analytics

Lecture 3: Boolean Retrieval and the Inverted Index

Institute of Technology of Cambodia · Department of Applied Mathematics and Statistics

Today

- **Boolean retrieval**
  - The term-document incidence matrix and the inverted index
  - Processing Boolean queries: the merge; query optimization
- **Enhanced postings lists**
  - Faster merges: skip pointers
  - Phrase queries: biword index and positional index
- **Limits of Boolean retrieval**

Boolean retrieval

Information Retrieval (IR)

Finding materials (usually documents)

of an **unstructured** nature (usually text)

that satisfy an **information need**

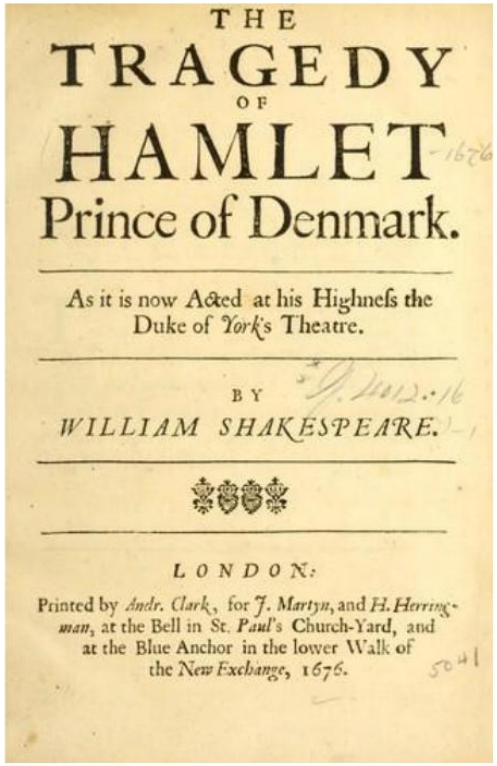
from within **large collections** (usually stored on computers).

Let's start with Boolean Retrieval with Shakespeare!

Boolean retrieval with Shakespeare



- **The collection**: ~40 plays by Shakespeare
  - e.g. https://shakespeare.mit.edu/index.html



THE
TRAGEDY
OF
HAMLET
Prince of Denmark.

As it is now Acted at his Highneſs the Duke of *York*'s Theatre.

BY
WILLIAM SHAKESPEARE.

LONDON:

Printed by *Andr. Clark*, for *J. Martyn*, and *H. Herringman*, at the Bell in St. *Paul*'s Church-Yard, and at the Blue Anchor in the lower Walk of the *New Exchange*, 1676.

Enter HAMLET

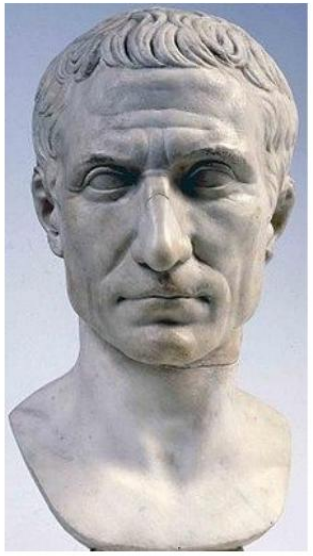
HAMLET

To be, or not to be, that is the question,
Whether 'tis nobler in the mind to suffer
The slings and arrows of outrageous fortune,
Or to take arms against a sea of troubles,

Boolean retrieval with Shakespeare

- **The information need** (in verbal form): which plays of Shakespeare mention ***Brutus*** *and* ***Caesar*** but *not* ***Calpurnia***?













Boolean retrieval with Shakespeare

- **The information need**: which plays mention ***Brutus*** and ***Caesar*** but not ***Calpurnia***?
- **3 conditions** to be satisfied at the same time
    1. Mentions Brutus
    2. Mentions Caesar
    3. Does not mention Calpurnia
- **The query**: Brutus AND Caesar AND (NOT Calpurnia)

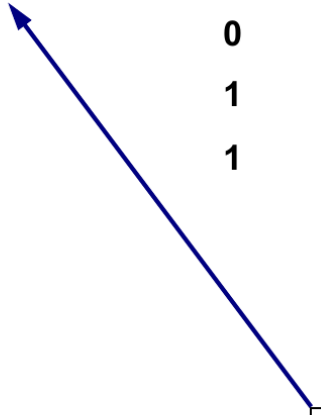
Boolean retrieval with Shakespeare

- **The query**: Brutus AND Caesar AND (NOT Calpurnia)
- **Naïve approach**:
  - For each play, run CTRL+F for **Brutus**, **Caesar** and **Calpurnia**, separately
  - If there is **at least one** match for Brutus, **at least one** for Caesar, but **none** for Calpurnia, add the play to the result
- It's one solution, but why isn't it the only solution?
  - **Too slow!** (for large corpora)

Indexing

- **The index**: term-document incidence matrix

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0



1 if the play contains the word, 0 otherwise

Query processing

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0

Brutus AND Caesar AND (NOT Calpurnia)

- Take the rows for ***Brutus, Caesar*** and ***Calpurnia*** (complemented, why?) and bitwise *AND* them.

```
110100 AND 110111 AND 101111 = 100100
```

Results

- ## Antony and Cleopatra, Act III, Scene ii

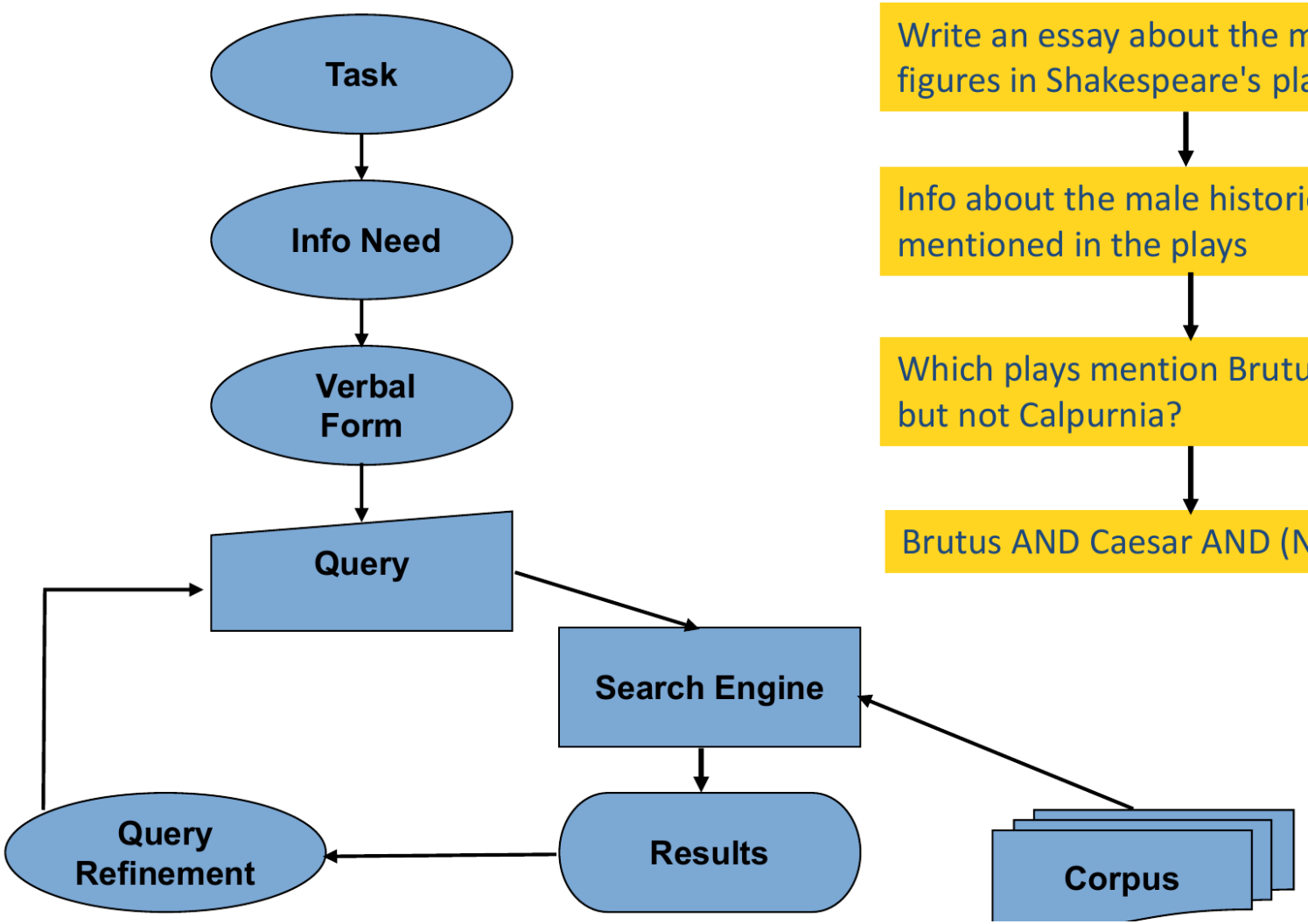
Agrippa [Aside to DOMITIUS ENOBARBUS]: Why, Enobarbus,
When Antony found Julius ***Caesar*** dead,
He cried almost to roaring; and he wept
When at Philippi he found ***Brutus*** slain.

- ## Hamlet, Act III, Scene ii

Lord Polonius: I did enact Julius ***Caesar*** I was killed i' the Capitol; ***Brutus*** killed me.



The classic search model



Write an essay about the male historical figures in Shakespeare's plays

Info about the male historical figures mentioned in the plays

Which plays mention Brutus and Caesar but not Calpurnia?

Brutus AND Caesar AND (NOT Calpurnia)

Relevance is the key!

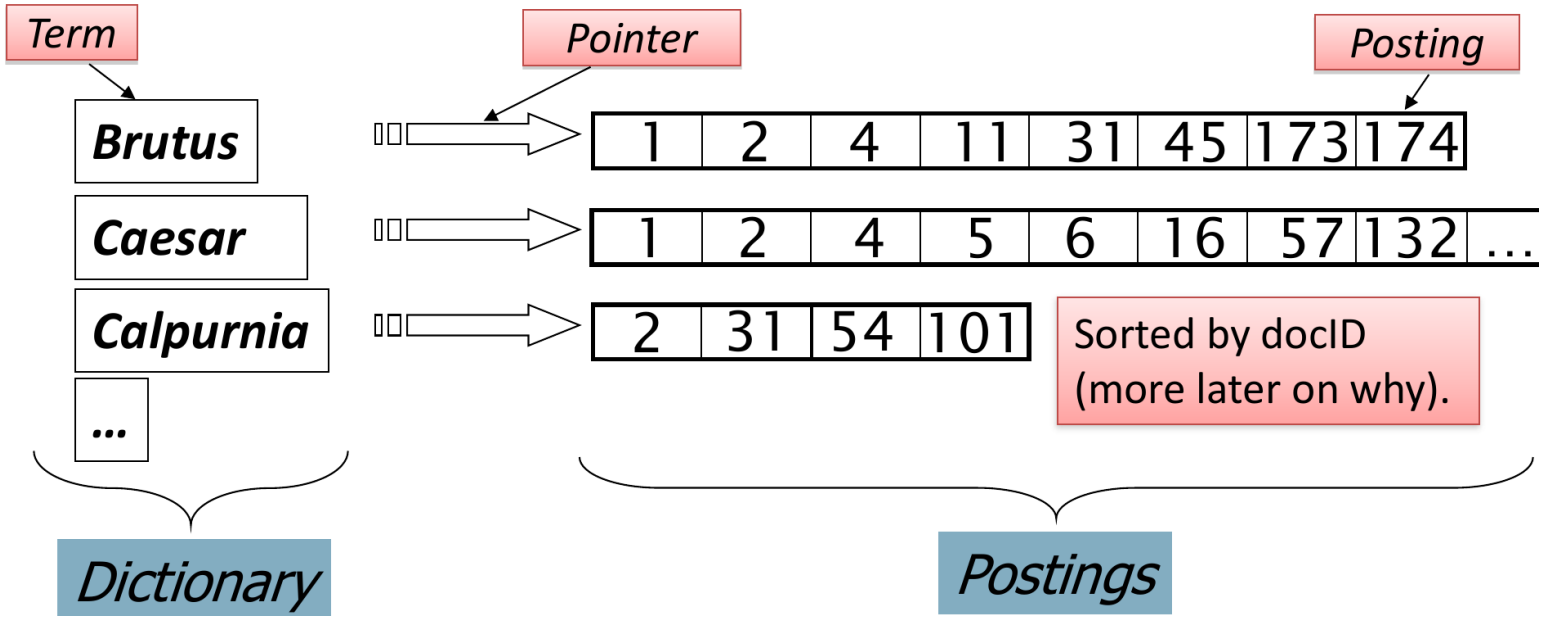
- Information Retrieval (IR) is **finding material** (usually documents) … that satisfies an **information need** …
- **Relevance**: whether the documents returned help to **satisfy** the information need.
- Evaluation metrics (covered in week 6)
  - **Precision**: fraction of retrieved docs that are **relevant** to the user's information need
  - **Recall**: fraction of **relevant** docs in the collection that are retrieved

Bigger collections: tough to build the matrix

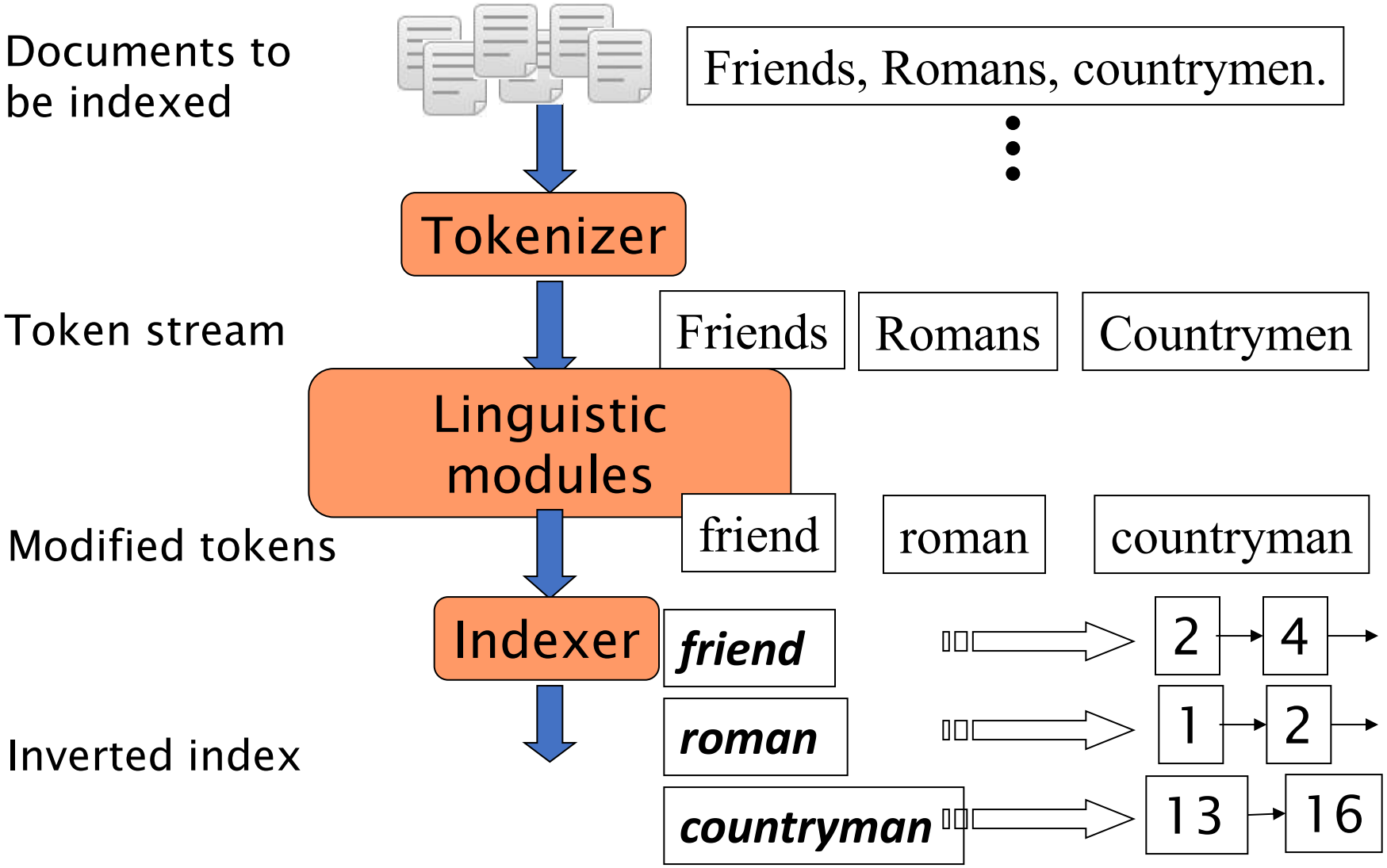
- Consider ***N*** = 1 million documents, each with about 1000 words
  - 1000 × 1 million = 1 billion words; ~6 bytes/word → **6 GB** of data
- Say there are ***M*** = 500K *distinct* terms among these
- 500K × 1M matrix has **half a trillion** 0s and 1s. B-I-G
- But it is **extremely sparse**
  - at most **1K** 1s among the **500K** cells in each **column** (document)
- Can we do better?

Inverted index

- Observation 1: identify each document by a serial number, the **docID**
  - Antony and Cleopatra → 1, Julius Caesar → 2, …
- Observation 2: searches start with terms
  - for each term *t*, store a **list** of all documents that contain *t*



Inverted index construction



Indexer steps: generate token sequence

- Generate sequence of **(Term, DocID)** pairs

Doc 1

I did enact Julius Caesar I was killed i' the Capitol; Brutus killed me.

Doc 2

So let it be with Caesar. The noble Brutus hath told you Caesar was ambitious



Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

Indexer steps: sort

- Sort by **terms**
  - and then **docID**

Core indexing step

Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2



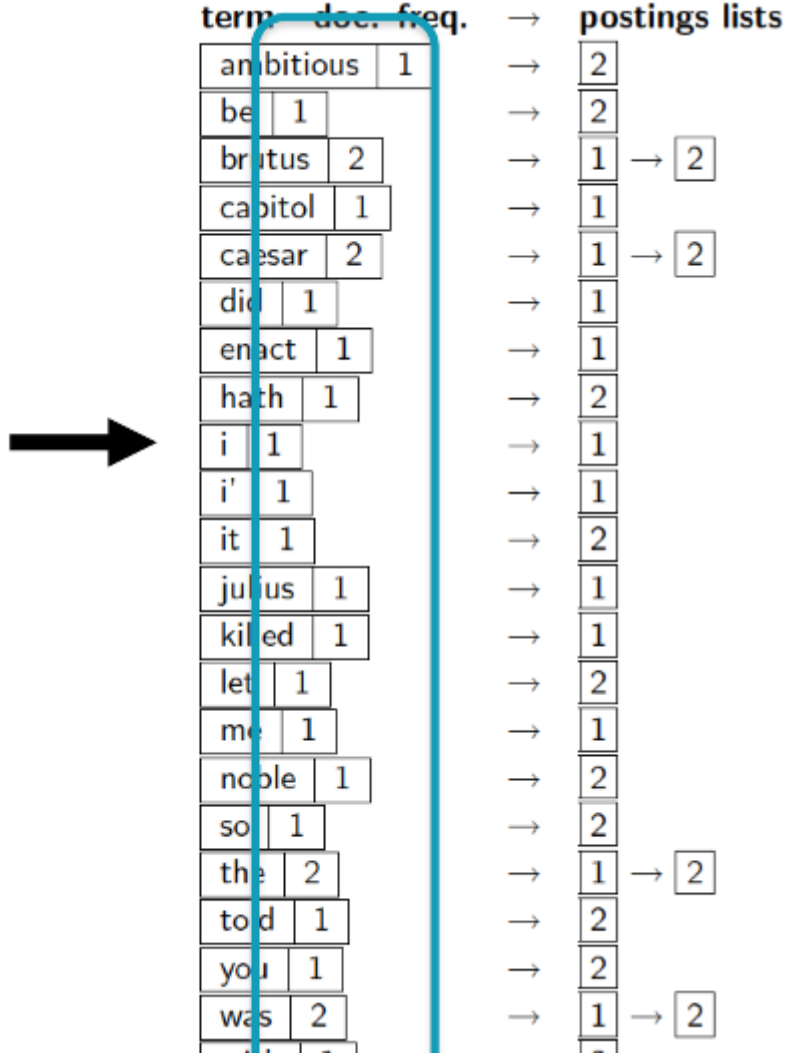
Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2

Indexer steps: dictionary & postings

- Multiple term entries in a single document are **merged**
- Split into **Dictionary** and **Postings**
- **Document frequency** information is also stored

Why frequency?
Will discuss later.

Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2



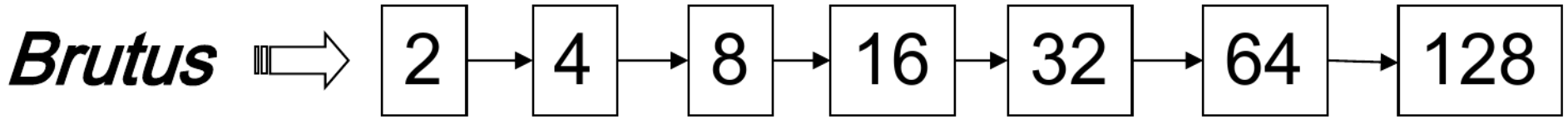
How do we process queries?

- Boolean queries
  - AND
  - OR
  - NOT
- Basic query optimization

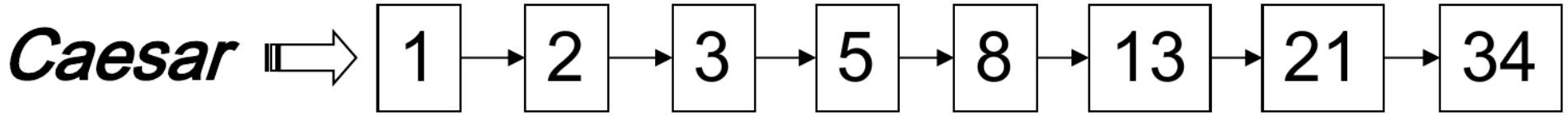
Later today:
• How to further optimize query processing?
• What other kinds of queries can we process?

Query processing: AND

- How to process this query? ***Brutus*** AND ***Caesar***
  - Locate ***Brutus*** in the Dictionary; retrieve its postings



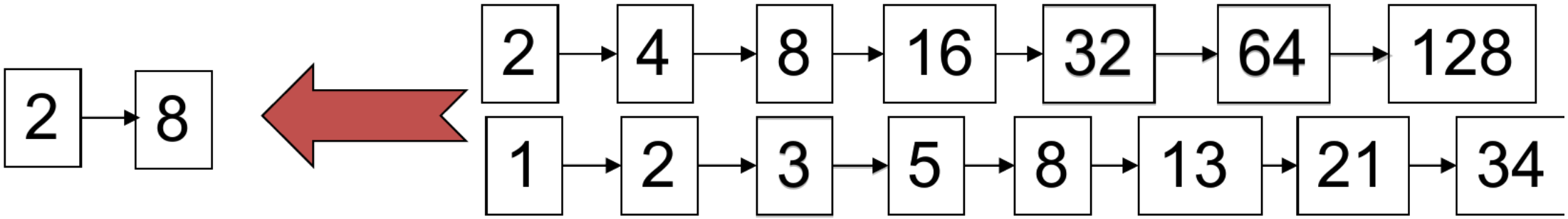
  - Locate ***Caesar*** in the Dictionary; retrieve its postings



- "Merge" the two postings
  - (Intersect) keep only the common entries

The merge

- Walk through the two postings simultaneously, in time linear in the total number of postings entries



If the list lengths are *x* and *y*, the merge takes O(*x* + *y*) operations.

Crucial: postings must be **sorted by docID**.

Intersecting two postings lists (a "merge" algorithm)

```
INTERSECT(p1, p2)
 1  answer ← ⟨ ⟩
 2  while p1 ≠ NIL and p2 ≠ NIL
 3  do if docID(p1) = docID(p2)
 4        then ADD(answer, docID(p1))
 5             p1 ← next(p1)
 6             p2 ← next(p2)
 7        else if docID(p1) < docID(p2)
 8                then p1 ← next(p1)
 9                else p2 ← next(p2)
10  return answer
```

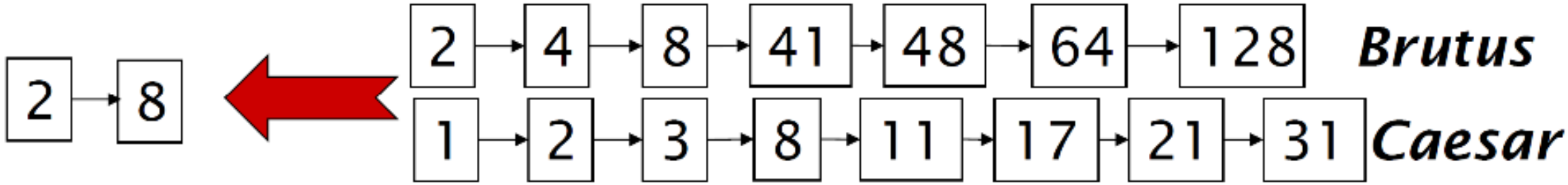
Query processing: OR and NOT

- ***Brutus*** OR ***Caesar***
  - Locate both terms in the Dictionary; retrieve their postings
  - "Merge" the two postings: **(Union)** keep all entries that appear in any of the two postings
- NOT ***Brutus***
  - Retrieve the **full list** of documents
  - Locate ***Brutus*** in the Dictionary; retrieve its postings
  - "Merge": **(Filtering)** keep entries in the full list but **not** in the postings

Faster merges: skip pointers

Recall basic merge

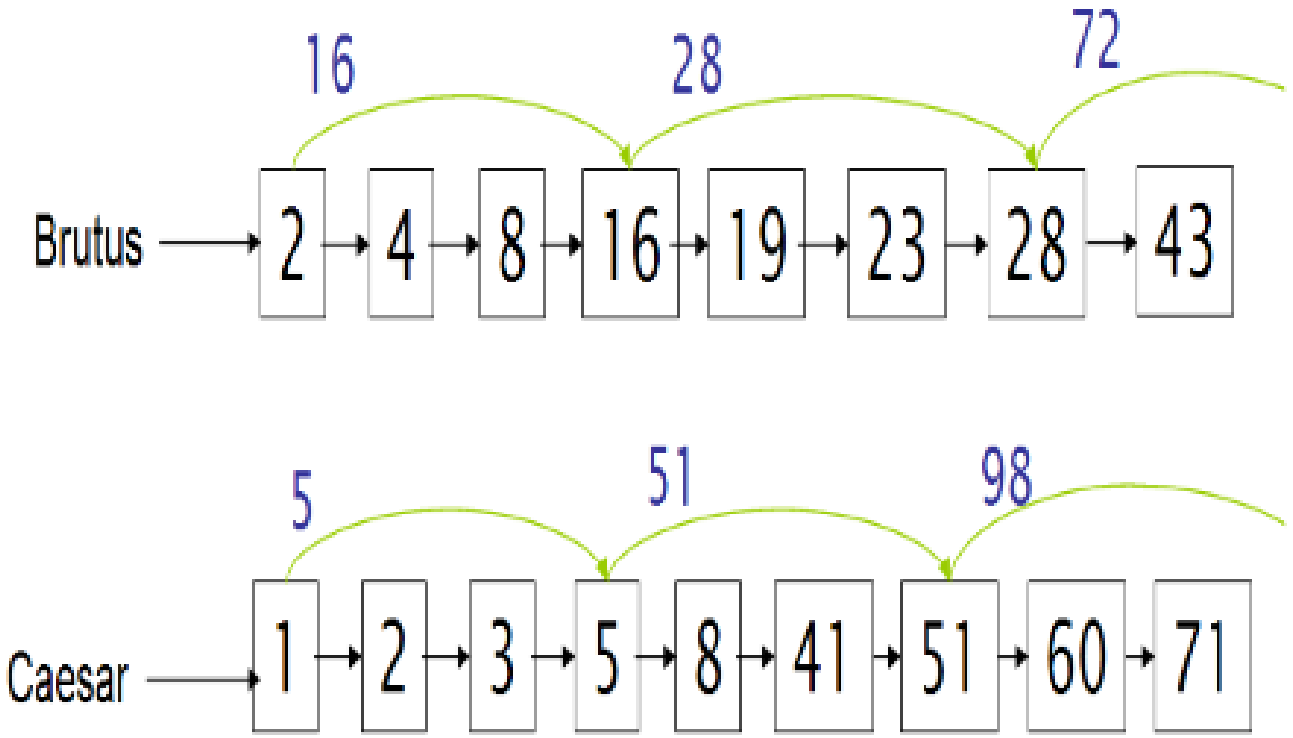
- Walk through the two postings simultaneously, in time linear in the total number of postings entries



If the list lengths are *m* and *n*, the merge takes O(*m* + *n*) operations.

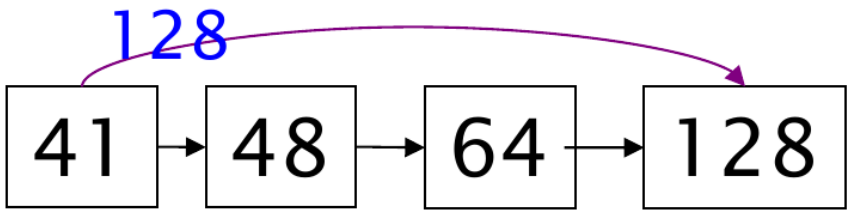
Can we do better?

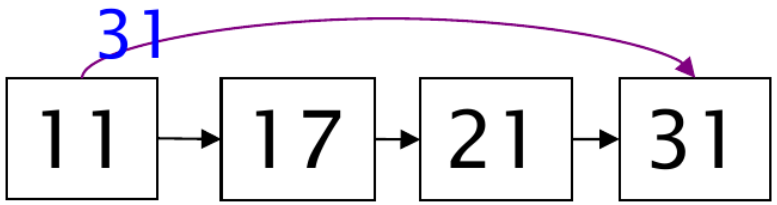
Adding skip pointers to postings

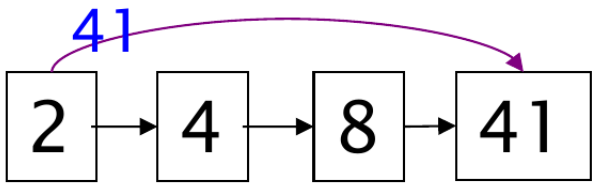


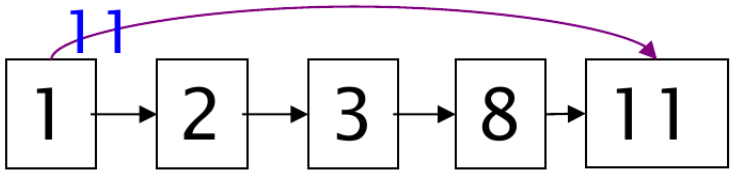
- To **skip postings** that are not part of the results

Query processing with skip pointers





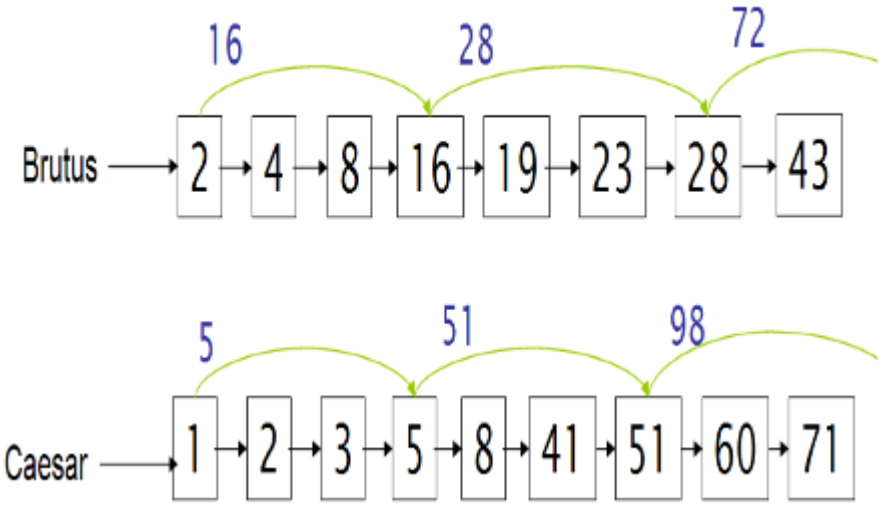




- Looking at **41** (upper) and **11** (lower)
  - skip successor of 11 is **31** ≤ 41
  - safe to skip to 31 directly. (Why?)
- Looking at **2** (upper) and **1** (lower)
  - skip successor of 1 is **11** > 2
  - NOT safe to skip! (Why?) Advance to 2

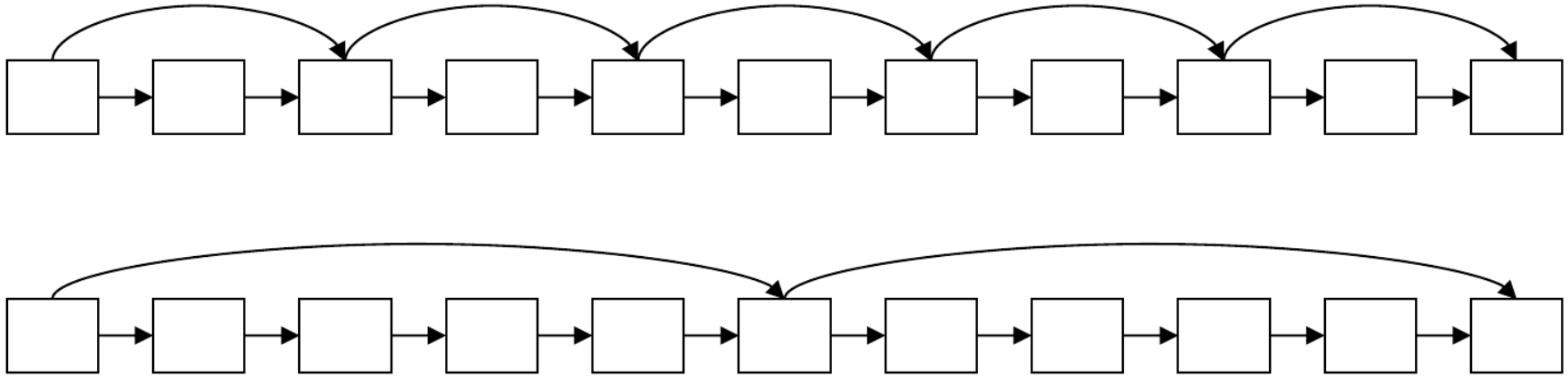
Query processing with skip pointers

```
INTERSECTWITHSKIPS(p1, p2)
 1  answer ← ⟨ ⟩
 2  while p1 ≠ NIL and p2 ≠ NIL
 3  do if docID(p1) = docID(p2)
 4        then ADD(answer, docID(p1))
 5             p1 ← next(p1)
 6             p2 ← next(p2)
 7        else if docID(p1) < docID(p2)
 8                then if hasSkip(p1) and (docID(skip(p1)) ≤ docID(p2))
 9                     then while hasSkip(p1) and (docID(skip(p1)) ≤ docID(p2))
10                          do p1 ← skip(p1)
11                     else p1 ← next(p1)
12                else if hasSkip(p2) and (docID(skip(p2)) ≤ docID(p1))
13                     then while hasSkip(p2) and (docID(skip(p2)) ≤ docID(p1))
14                          do p2 ← skip(p2)
15                     else p2 ← next(p2)
16  return answer
```



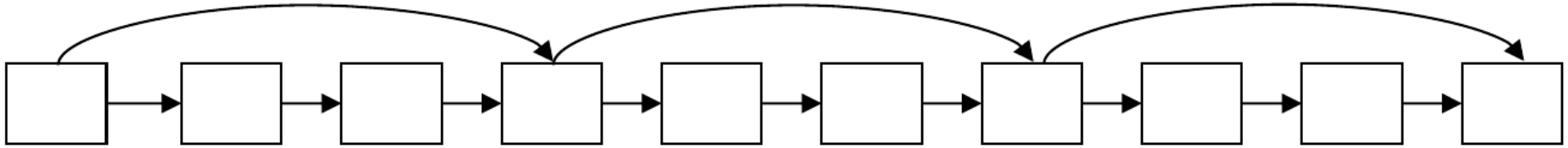
Adding skip pointers to postings

- Part of the **indexing** process
- More skips or fewer skips?
  - More skips → shorter skip spans ⇒ more likely to skip. But lots of comparisons to skip pointers
  - Fewer skips → few pointer comparisons, but long skip spans ⇒ few successful skips



Placing skips

- Simple heuristic: for postings of length L, use $\sqrt{L}$ **evenly-spaced skip pointers**
  - this **ignores** the **distribution of docIDs**



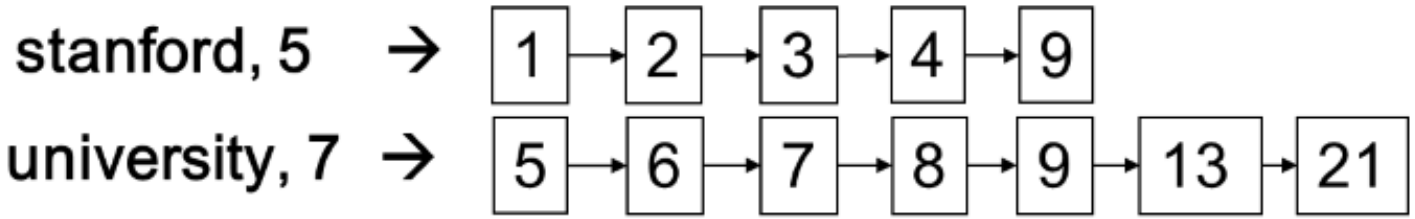
- This definitely used to help; but we need to be aware of the cost!
  - Pointer comparison
  - Disk space and I/O time for storing and loading a bigger list
  - Updating of pointers in a dynamic list

Handling phrase queries

Phrase queries

- Want to be able to answer queries such as "***stanford university***" – as a phrase
  - Not the same as stanford AND university
  - Popular and easy to understand
  - E.g., *"I went to Stanford University"* is a match, but *"I went to university at Stanford"* is not.

- Not suffice to store individual terms with the docIDs.



A first attempt: Biword indexes

- Biword indexes: an information retrieval data structure that indexes consecutive pairs of words as a single vocabulary term to handle two-word phrase queries.
  - *Example "I went to Stanford University"* → 4 biwords: ***I went, went to, to Stanford, Stanford University***

- **Process** two-word phrase queries by looking up the biwords directly
  - *stanford university*, 1 → 9
- **Longer** phrases: a Boolean query on biwords
  - *"stanford university palo alto"* → ***stanford university*** AND ***university palo*** AND ***palo alto***

- There could be false positives… **Why?**

Noted: a false positive is a document the system returns that it should not have returned

Issues for biword indexes

- False Positives: Breaking long phrases into biword intersections can match documents where the word pairs appear in different parts of the text rather than as a contiguous phrase.
- Index Blowup: Storing all unique word pairs significantly expands the dictionary size compared to a basic single-term index
- Not the standard solution, but can be part of a **compound strategy**

Solution 2: Positional indexes

- In the postings, store for each ***term*** the position(s) in which tokens of it appear:

<***term***, document frequency;
doc1: position1, position2 … ;
doc2: position1, position2 … ;
etc.>

<***be***: 993427;
1: 7, 18, 33, 72, 86, 231;
2: 3, 149;
4: 17, 191, 291, 430, 434;
5: 363, 367, …>

Quick check:
Which of docs 1,2,4,5 could contain "***to be or not to be***"?

Processing a phrase query

- Query: “to be or not to be”

- Retrieve the lists for each term
- Merge at the document level and then position level.
  - ***to**: 2*:1,17,74,222,551; *4*:8,16,190,429,433; *7*:13,23,191; ...
  - ***be**: 1*:17,19; *4*:17,191,291,430,434; *5*:14,19,101; ...
- Need to deal with more than just equality / intersection...

Boolean retrieval in practice

Boolean retrieval

- The Boolean retrieval model is able to process queries which are based on Boolean expressions:
  - Views each document as a set of words
  - Is precise: document matches condition or not.
- Perhaps the simplest model to build an IR system on

- Primary commercial retrieval tool for 3 decades.
- Many search systems you still use are Boolean:
  - e.g. library catalogues

Example: Westlaw

http://www.westlaw.com/

- Largest commercial (paying subscribers) legal search service (started 1975; ranking added 1992)
  - Tens of terabytes of data; 700,000 users
- Long, precise queries; proximity operators; incrementally developed; not like web search
  - What is the statute of limitations in cases involving the federal tort claims act?
    LIMIT! /3 STATUTE ACTION /S FEDERAL /2 TORT /3 CLAIM
    - /3 = within 3 words, /S = in same sentence
- Many professional searchers still like Boolean search
  - You know exactly what you are getting
- But that doesn't mean it actually works better…

Limits of Boolean retrieval

- No Result Ranking
  - All-or-nothing matching: Documents either match the Boolean expression completely or not at all.
  - Lack of relevance order: There is no grading or scoring system to put the most useful documents at the top; every matched document is treated as equally relevant.
  - Extreme result sizes: Using **AND** operators too strictly can result in zero hits, while using **OR** operators can flood the user with thousands of unranked documents.
- Most users can't (or won't) write Boolean queries
- Next week: **ranked retrieval**: vector space model and TF-IDF



Thank you!